we defined. To do this in Jest, we can call the `expect` function and pass in the actual data received from the consumer. We then use the `toEqual` matcher and pass in the expected data:

```
await provider.executeTest(async mockProvider => {
    const movies = await fetchMovies(mockProvider.url);
    expect(movies[0]).toEqual(EXPECTED_BODY);
});
```

For your reference, the full test should look like this one:

```
const path = require('path');
const { fetchMovies } = require('./consumer');
const { PactV3, MatchersV3 } = require('@pact-foundation/pact');

const provider = new PactV3({
  dir: path.resolve(process.cwd(), 'pacts'),
  consumer: 'WebConsumer',
  provider: 'MoviesAPI',
});
const EXPECTED_BODY = { id: 1, name: "My movie", year: 1999 }

describe('Movies Service', () => {
  describe('When a GET request is made to /movies', () => {
    test('it should return all movies', async () => {
      provider
        .uponReceiving('a request to all movies')
        .withRequest({
          method: 'GET',
          path: '/movies',
        })
        .willRespondWith({
          status: 200,
          body: MatchersV3.eachLike(EXPECTED_BODY),
        });

      await provider.executeTest(async mockProvider => {
        const movies = await fetchMovies(mockProvider.url);
        expect(movies[0]).toEqual(EXPECTED_BODY);
      })
    });
  });
});
```

Now, you’re ready to run the test! To run this test, let’s create a new script in our package.json called `test:web:consumer`, which uses the `jest` command followed by the test file you want to execute:

```
"scripts": {
    "start:provider": "cross-env PORT=3001 node provider/provider-service.js",
    "test:web:consumer": "jest consumer-web/consumer-contract.spec.js"
  }
```